QA ENGINEER

Technical Challenge Submission

Website Test Strategy, Framework Analysis and Optional Enhancements

POONAM AWASTHI

QA Engineer II

Starting repository: Velir.QA.TakeHome

Submission note

Framework observations are based on the supplied challenge and the public repository structure/documentation. Test execution results should be added after running the suite locally.

Executive Summary

The application supports a hotel room booking journey. The most business-critical path is the successful creation of a valid reservation, while the highest operational risk is inaccurate availability or duplicate booking behavior. The proposed approach therefore priorities risk-based coverage of booking creation, validation, availability and API behavior before broader non-functional testing.

Recommended focus

Stabilise the core booking flow first, then expand negative API coverage, conflict handling, accessibility checks, test data quality and continuous integration.

Delivery Overview

Section	What is covered
1. Website Test Strategy	Product analysis, critical paths, priority workflows, failure states and test types
2. Framework Analysis	Current UI/API coverage, gaps, architecture, quality assessment and recommendations
3A. Missing Tests	Three concrete Playwright test candidates with implementation guidance
3B. AI-Assisted Enhancement	A practical coverage-review prompt and a proposed pull-request workflow
3C. Other Improvements	Accessibility, realistic test data, CI and reporting examples

Assumptions and Scope

- The website and repository may evolve after this document is prepared.
- Selectors, endpoint paths, helper signatures, and expected error messages in sample code must be aligned with the checked-out repository before execution.
- The examples are intentionally concise and are designed to demonstrate test intent and framework extension points.

1. Website Test Strategy

1.1 Product Analysis

The product is a hotel booking application. Its principal customer value is enabling a visitor to identify a suitable room, provide guest and stay details, submit the reservation, and receive a clear outcome.

Core Functionality

- Display room information and pricing
- Allow users to choose booking dates
- Capture guest contact information
- Create and retrieve bookings
- Communicate validation errors and successful confirmation

Critical User Journey

Primary path
Room selection > date selection > guest details > submission > booking confirmation

This journey should be treated as the smoke path because failure prevents the product from completing its main commercial purpose.

1.2 Key Features and Success Criteria

Priority	Feature / workflow	Success looks like
P0	Room booking	A valid booking is created, and a clear confirmation is shown.
P0	Availability and date handling	Only valid, available date ranges can be selected, and conflicting bookings are prevented.
P1	Guest data validation	Required fields are enforced, and invalid values produce clear, useful feedback.
P1	Room information	Room description, capacity, amenities, and price are accurate and understandable.
P1	Booking API	Supported operations return correct status codes, schema, and persisted data.

1.3 Failure States and Edge Cases

Area	Examples	Expected behaviour
Dates	Past date; same-day stay; checkout before check-in; long stay	Reject invalid ranges consistently and explain the issue.
Guest details	Blank name; invalid email; short/long phone; whitespace-only values	Prevent submission and identify the affected field.
Availability	Already-booked room; two simultaneous	Prevent overbooking and return a deterministic conflict

Area	Examples	Expected behaviour
	requests; stale availability	response.
Resilience	Slow API; network interruption; server error; retry	Do not create duplicate bookings; preserve or clearly reset user input.
Security	Script payload; injection-like input; oversized payload	Input is safely handled and no executable content is reflected.
Usability	Keyboard-only flow; zoom; mobile viewport	Core booking remains operable and readable.

1.4 Risk-Based Test Order

1. End-to-end successful booking, because it proves the primary commercial path.
2. Date and required field validation, because invalid data can corrupt inventory or customer records.
3. Availability and duplicate-booking protection, because failures can create operational conflicts.
4. API contract and negative behaviour because UI reliability depends on predictable services.
5. Accessibility, browser coverage, performance and security to broaden product confidence after the core flow is stable.

1.5 Recommended Test Types

Test type	Purpose	Examples
Smoke	Rapid confidence in critical path	Home page loads; valid booking succeeds.
Functional UI	Validate user-facing behaviour	Fields, dates, messages, navigation.
API	Validate service contracts and errors	Create/get/update/delete, schema, invalid payloads.
Integration	Verify components work together	UI submission persists and can be retrieved.
Accessibility	Detect barriers	Keyboard, labels, focus, contrast, landmarks.
Security	Reduce input and access risks	Sanitisation, auth checks, payload limits.
Performance	Assess responsiveness and stability	API latency, concurrency, booking load.
Compatibility	Check supported environments	Chromium, Firefox, WebKit and target viewports.

2. Framework Analysis

This section reflects the public repository documentation and visible project structure. It does not claim that the suite was executed in this document-generation environment.

2.1 Current Coverage

The repository describes a Playwright and TypeScript project with UI and API tests, a page object for the booking page, API utilities and test-data utilities.

Layer	Covered scenarios
UI - positive	Create a booking with valid details and verify a success confirmation.
UI - negative	Attempt booking without a required phone number and verify browser validation prevents submission.
API - read collection	Retrieve bookings and validate response structure/data integrity.
API - read single	Retrieve a booking by identifier and verify booking details.
API - create	Create a booking and verify returned details.

2.2 Coverage Gaps

- Invalid and boundary date combinations
- Negative API payloads and explicit error-contract assertions
- Booking update and deletion behaviour
- Conflict and duplicate-booking handling
- Cross-layer verification that a UI-created booking is retrievable via API
- Accessibility scanning and keyboard interaction
- Resilience, performance, security and visual regression coverage

2.3 Strategy-to-Automation Gap

Capability	Priority	Observed coverage	Recommended action
Successful booking	P0	Covered	Retain as smoke test; strengthen data assertions.
Field validation	P0/P1	Partial	Add parameterized required field and format cases.
Date logic	P0	Not documented	Add boundary and invalid-range tests.
Availability/conflicts	P0	Not documented	Add API-first conflict tests and one UI check.
API negative paths	P1	Not documented	Add schema, missing-field and invalid-value cases.
Accessibility	P1	Not documented	Add automated scans plus targeted keyboard tests.
CI/reporting	P1	Repository-dependent	Publish artefacts and gate pull requests.

2.4 Framework Quality

What works well

- Clear separation between UI tests, API tests, pages and utilities supports discoverability.
- A Page Object Model reduces direct page interaction details in test cases.
- Dedicated API and test-data utilities provide reasonable extension points.

What I would improve

- Prefer role, label or test-id locators over fragile CSS selectors.
- Make test data unique and deterministic; record created booking IDs for cleanup.
- Use API setup for UI tests where it shortens execution without weakening the intent.
- Add explicit response schema and business-data assertions.
- Create smoke/regression/accessibility projects or tags.
- Capture traces, screenshots and HTML reports on failure.
- Run linting and tests in continuous integration.

2.5 Next Three Changes

Order	Change	Reason
1	Add negative API and date-validation coverage	High risk, fast feedback and easier diagnosis.
2	Add conflict handling and reliable test-data lifecycle	Protects inventory integrity and reduces test collisions.
3	Add accessibility checks and CI artefacts	Broadens quality signals and makes failures reviewable.

3. Optional Enhancements (Bonus)

3A. Implement Missing Tests

Implementation note
The following examples are submission-ready templates. Before running them, map placeholder methods, selectors, endpoint paths and expected messages to the checked-out repository.

Test 1: Reject an Invalid Email Address

Risk addressed: poor contact data can prevent booking communication. The test verifies that an invalid email cannot be submitted and that the user receives field-level feedback.

Suggested file: tests/ui/invalid-email.spec.ts

```
import { test, expect } from '@playwright/test';
import { HotelBookingPage } from '../pages/HotelBookingPage';

test('[NEGATIVE] rejects an invalid email address', async ({ page }) => {
  const bookingPage = new HotelBookingPage(page);
  await bookingPage.open();

  await bookingPage.fillGuestDetails({
    firstName: 'Asha',
    lastName: 'Sharma',
    email: 'invalid-email',
    phone: '9876543210'
  });
  await bookingPage.submitBooking();

  const email = page.getByLabel(/email/i);
  await expect(email).toBeInvalid();
});
```

Test 2: Reject Checkout Before Check-in

Risk addressed: invalid date ranges can corrupt availability logic. The assertion should target the exact validation behaviour implemented by the application.

Suggested file: tests/ui/invalid-date-range.spec.ts

```
import { test, expect } from '@playwright/test';
import { HotelBookingPage } from '../pages/HotelBookingPage';

test('[NEGATIVE] rejects checkout before check-in', async ({ page }) => {
  const bookingPage = new HotelBookingPage(page);
  await bookingPage.open();

  await bookingPage.selectDates({
    checkIn: '2026-12-20',
    checkOut: '2026-12-15'
  });
  await bookingPage.submitBooking();

  await expect(page.getByText(/invalid|date range|checkout/i)).toBeVisible();
});
```

Test 3: Prevent a Conflicting Booking

Risk addressed: overbooking. The test creates an initial reservation and submits a second reservation for the same room and dates. Replace the expected status with the application contract discovered during execution.

Suggested file: tests/api/booking-conflict.spec.ts

```
import { test, expect } from '@playwright/test';
import { ApiHelper } from '../utils/ApiHelper';
import { createBookingData } from '../utils/TestDataGenerator';

test('[API] prevents a conflicting booking', async ({ request }) => {
  const api = new ApiHelper(request);
  const booking = createBookingData({
    roomId: 1,
    checkIn: '2026-12-20',
    checkOut: '2026-12-22'
  });

  const first = await api.createBooking(booking);
  expect(first.ok()).toBeTruthy();

  const duplicate = await api.createBooking(booking);
  expect(duplicate.status()).toBeGreaterThanOrEqual(400);
});
```

Test Design Notes

- Use unique future dates where the environment permits, or seed/clean data through supported APIs.
- Avoid asserting generic text such as “Invalid” when a stable error code or accessible error element is available.
- Keep one assertion focused on the main behavioral outcome and add diagnostic assertions only where useful.
- If the product intentionally permits multiple bookings, replace the conflict test with an explicit availability/business-rule test.

3B. AI-Assisted Enhancement

Use Case: Pull-Request Coverage Review

An AI-assisted review can compare changed application or test files with the existing suite, identify candidate risks and propose tests for human review. AI output should remain advisory and must not merge or execute unreviewed code.

> Guardrail: Do not send secrets, production data, personal data or unsupported proprietary content to an AI service. Follow the organization's approved tooling and data handling rules.

Reusable Prompt

AI review prompt

```
You are supporting a senior QA automation engineer reviewing a Playwright TypeScript project.

Inputs:
- Changed files from the pull request
- Existing tests and page/API helpers
- The product's critical booking journey and validation rules

Tasks:
1. Summarise the user-visible behaviour changed by the pull request.
2. Identify missing positive, negative, boundary, accessibility and API-contract tests.
3. Rank each gap as P0, P1 or P2 using business impact and likelihood.
4. Identify likely flakiness risks such as timing waits, shared data or unstable selectors.
5. Propose up to three Playwright TypeScript tests for the highest-priority gaps.
6. Reuse existing fixtures, page objects and helper methods where possible.
7. Mark all assumptions explicitly. Do not invent selectors, endpoints or expected messages.

Output format:
- Change summary
- Risk-ranked coverage gaps
- Proposed tests with rationale
- Assumptions requiring human confirmation
```

3C. Other Improvements

Accessibility Scan with axe-playwright

Automated accessibility scanning is useful as a repeatable baseline but should be supplemented by keyboard and screen-reader-oriented checks.

Suggested file: tests/ui/accessibility.spec.ts

```
import { test, expect } from '@playwright/test';
import { AxeBuilder } from '@axe-core/playwright';

test('booking page has no automatically detectable critical violations', async ({ page }) => {
  await page.goto('/');

  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa'])
    .analyze();

  const critical = results.violations.filter(
    violation => violation.impact === 'critical'
  );
  expect(critical).toEqual([]);
});
```

Realistic Test Data

Example extension for TestDataGenerator.ts

```
import { faker } from '@faker-js/faker';

export function createGuestData() {
  return {
    firstName: faker.person.firstName(),
    lastName: faker.person.lastName(),
    email: faker.internet.email(),
    phone: faker.string.numeric(10)
  };
}
```

Continuous Integration

Suggested file: .github/workflows/playwright.yml

```
name: Playwright Tests

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```

4. Execution and Submission Checklist

Run Locally

```
git clone https://github.com/Velir/Velir.QA.TakeHome.git
cd Velir.QA.TakeHome
npm ci
npx playwright install
npx playwright test
npx playwright show-report
```

5. Conclusion

The existing project provides a compact foundation for UI and API automation. The highest-value expansion is to add negative date and payload coverage, protect availability integrity, improve test-data lifecycle management, automate accessibility checks and run the suite with reviewable CI artefacts. The bonus proposals demonstrate practical automation design while keeping assumptions visible and human review in control.